VL-JEPA: ajuste fino v3

Banco de referencias factorizado, InfoNCE bidireccional y lado de texto entrenable.
Decisiones, justificación y referencias. Proyecto EL7006 · rama `vl-jepa` · 13-11-2026

Resumen

El run publicado de VL-JEPA (canónico, 1 época) aprende forma y posición horizontal, pero queda en 0,670 en posición vertical y 0,760 en color, donde los tres VLM autorregresivos superan 0,87. El análisis de pérdida de información del equipo ubicó el problema en el **decodificador**: la información del atributo llega a la salida del predictor ($\hat{s}$) y se pierde al leerla contra el banco de textos. Este documento describe el rediseño que sigue de ese hallazgo. Cambia cuatro cosas y deja intacto el resto del presupuesto común.

	Run publicado (etapa 2 reformulada)	Ajuste v3	Sección
Banco de referencias	504 captions canónicos completos	21 frases, una por valor de atributo	3
Consulta	El prompt p1 (una por imagen)	Una por atributo: forma, color, vertical, horizontal	3
Pérdida	InfoNCE en una dirección sobre el banco	InfoNCE bidireccional (directo sobre el banco + inverso sobre el lote)	4
Entrenable	LoRA en el predictor + vis_proj	LoRA en el predictor + vis_proj + out_proj + proyección del Y-Encoder (lr × 0,05)	5
Congelado	X-Encoder, Y-Encoder completo, out_proj	X-Encoder y transformer de texto del Y-Encoder	5
Épocas	1	1 (fila comparable) y 4 (presupuesto extendido, reportado aparte)	6
Lectura	Vecino más cercano entre 504 frases	Softmax sobre 3, 7 u 8 frases legibles, con probabilidad y margen en nats	3

Estado. El código está escrito y probado con modelos diminutos de arquitectura real en CPU (sección 8). **No se ha entrenado ni medido nada a escala real**: este equipo no tiene los pesos ni los runs del cluster. Los números del run publicado salen de `reports/tables`; todo lo que se afirma sobre el ajuste v3 es diseño e hipótesis hasta correr `configs/queues/stage2c_vljepa.txt`.

1. Punto de partida

Resultados en test (10 000 imágenes, objetivo canónico, prompt p1, mejor checkpoint en val) de `reports/tables/comparacion_test` y `eda_finetuning`:

Modelo	Forma	x	y	Posición (x,y)	Color	Color ±1	Paso elegido
VL-JEPA (open-vljepa, MSR-VTT)	1,000	0,956	0,670	0,644	0,760	0,977	1800 de 1954
Qwen3.5-0.8B Base	0,999	—	—	1,000	0,914	1,000	1800
InternVL3-1B	—	—	—	0,999	0,887	1,000	1000
LLaVA-OneVision 0.5B	—	—	—	0,999	0,876	1,000	800

Tres hechos orientan el rediseño. (i) VL-JEPA acierta la forma siempre, así que el predictor sí extrae información fina de la imagen. (ii) Falla en un atributo de tres valores (vertical) mientras acierta otro de tres valores (horizontal): no es un problema de capacidad general. (iii) Su mejor paso está casi al final de la época, mientras LLaVA-OV e InternVL3 saturan antes de la mitad: puede faltarle presupuesto, además de diseño.

2. Diagnóstico: dónde se pierde la información

2.1 El modelo como cadena de procesamiento

La imagen determina el atributo A (forma, color, y, x). VL-JEPA lo procesa en cadena: imagen → tokens del X-Encoder → ŝ (predictor) → respuesta (decodificador sobre el banco). Por la desigualdad de procesamiento de datos [13], la información sobre A no puede aumentar de una etapa a la siguiente:

$$H(A) \;\geq\; I(A;\,\text{vis}) \;\geq\; I(A;\,\hat{s}) \;\geq\; I(A;\,\text{respuesta})$$

Cada término se acota por abajo con un lector q(a | representación) evaluado en imágenes nuevas, porque I(A; Z) = H(A) − H(A | Z) ≥ H(A) − E[−log q(A | Z)]. Con q igual a una sonda lineal se obtiene la V-información de Xu et al. [5] (y es la sonda de Alain y Bengio [14] que el repositorio ya usa); con q igual al softmax sobre el banco se obtiene la cota InfoNCE [3, 4] con el crítico fijo $\langle \hat{s},\, s_a \rangle / \tau$. La diferencia entre la sonda sobre ŝ y el softmax del banco es lo que *está* en ŝ y el decodificador no lee. Eso es lo que mide `scripts/info_loss.py` (sección 7), por atributo y en nats.

2.2 Por qué un banco de frases completas pierde información

En el banco de 504 captions, «…is at the top-left…» y «…is at the mid-left…» comparten todas las palabras salvo una. Si sus embeddings tienen similitud coseno c, la mayor diferencia de logits que cualquier ŝ unitario puede poner entre ambas es

$$\max_{\|\hat{s}\|=1} \frac{\langle \hat{s},\, s_a - s_b \rangle}{\tau} \;=\; \frac{\|s_a - s_b\|}{\tau} \;=\; \frac{\sqrt{2\,(1-c)}}{\tau}$$

Con τ = 0,07, c = 0,99 da un margen máximo de 2,0 nats y c = 0,999 de 0,64 nats. Es una propiedad del banco, no del predictor: si las referencias que compiten están casi paralelas, ningún entrenamiento del predictor puede pasar la información por ese softmax, y además el gradiente de InfoNCE en esa dirección es pequeño frente al de los atributos bien separados. El repositorio ahora calcula esta cota y la **capacidad** de cada grupo de referencias (la información máxima que un predictor ideal transmitiría a través del softmax, `vlmfid.models.bank.group_capacity`).

Qué es hipótesis. Que el Y-Encoder ponga «top» y «mid» casi paralelos en frases completas es esperable (los codificadores de frases son anisotrópicos [15] y EmbeddingGemma [11] se entrena para similitud semántica, donde esas dos frases son casi sinónimas), pero **no está medido**: el valor de c sale de `bank_geometry.json` al correr el script sobre el run publicado. Si c resultara bajo, el mecanismo sería otro y habría que revisar la sección 3.

Corrección de una recomendación anterior. En la conversación propuse «centrar el banco» como arreglo sin reentrenar. Restar la media μ a las referencias no cambia el vecino más cercano por producto interno: $\langle \hat{s},\, s_j - \mu \rangle = \langle \hat{s},\, s_j \rangle - \langle \hat{s},\, \mu \rangle$, y el segundo término es igual para todo j. Solo cambiaría algo al renormalizar después de centrar. Tampoco ayuda marginalizar por atributo cuando el banco es un factorial completo y aditivo. Por eso el arreglo pasa por cambiar los objetivos (el banco y el lado de texto), no por posprocesar la recuperación.

3. Banco de referencias factorizado

El banco nuevo tiene una frase por valor de atributo: 21 en total. Al modelo se le hace una consulta por atributo y cada ŝ se compara solo con las frases de su atributo. El caption final se compone con la misma plantilla canónica de la etapa 2, así que pasa por el mismo extractor y las mismas métricas que los VLM.

Atributo	Consulta al predictor	Frases de referencia (K)	Techo InfoNCE
Forma	What is the shape of the object in the image?	The shape of the object is {teapot, hare, dragon, cow, armadillo, horse, head}. (7)	log 7 = 1,95 nats
Color	What is the color of the object in the image?	The color of the object is {red, orange, yellow, green, cyan, blue, purple, pink}. (8)	log 8 = 2,08 nats
Vertical	What is the vertical position of the object in the image?	The vertical position of the object is {top, mid, bottom}. (3)	log 3 = 1,10 nats
Horizontal	What is the horizontal position of the object in the image?	The horizontal position of the object is {left, center, right}. (3)	log 3 = 1,10 nats

3.1 Por qué este diseño

- **Es el uso que el paper reporta como fuerte.** VL-JEPA hace VQA discriminativo codificando las respuestas candidatas con el Y-Encoder y eligiendo la más cercana a $\hat{s}$, con la pregunta como consulta del predictor [1, §4.2]. Clasificar con textos candidatos es también la lectura de CLIP [6]. El run publicado usaba el modelo como recuperador de captions, un modo en que el checkpoint (entrenado con MSR-VTT) es más débil.
- **Ataca el mecanismo de la sección 2.2.** Las frases que compiten ya no comparten el resto del caption: difieren en la única palabra que importa y la consulta le dice al predictor qué atributo se pregunta. Si aun así quedaran pegadas, la proyección entrenable del Y-Encoder (sección 5) puede separarlas, porque son 21 puntos y no 504.
- **Es explicable por construcción.** Cada decisión es un softmax sobre 3, 7 u 8 frases que se pueden leer. Cada predicción guarda `attr_pred`, `attr_prob` (probabilidad de cada candidato) y `attr_margin_nats` (ventaja sobre el segundo). Un error deja de ser «recuperó otra frase» y pasa a ser «dijo mid con 0,48 frente a top con 0,45».
- **La información se lee contra su techo.** Con K candidatos la cota InfoNCE llega a log K, que es la entropía máxima del atributo. La pérdida de validación se convierte directamente en nats entregados.
- **Sigue siendo lenguaje.** Las referencias son frases codificadas por el Y-Encoder, no vectores libres. Un banco de 21 vectores entrenables daría un clasificador lineal y dejaría de ser VL-JEPA.

3.2 Alternativas descartadas

Alternativa	Por qué no
Mantener los 504 captions y solo descongelar el texto	Sigue siendo un softmax de 504 clases donde los negativos difíciles difieren en una palabra, y la lectura sigue sin ser explicable por atributo. Queda como comparación: es el run publicado.
Captions originales de M3DI (~10 900)	Es el experimento que prueba la tesis del paper (el embedding absorbe plantillas y paletas), pero mezcla dos preguntas. Conviene correrlo después, con el decodificador ya resuelto.
Centrar o blanquear el banco sin reentrenar	No cambia el vecino más cercano (nota de la sección 2.2).
Cabeza lineal de clasificación sobre $\hat{s}$	Es la sonda: mide el techo, pero no decodifica a texto. Se reporta como cota superior en scripts/info_loss.py.
Las cuatro consultas por imagen en cada paso	Cuadruplica las secuencias del predictor (256 por lote). Se prefirió rotar una consulta por imagen y usar más épocas (sección 6).

4. Pérdida: InfoNCE bidireccional, como en el paper

El paper entrena el predictor y el Y-Encoder en conjunto con InfoNCE bidireccional [1, §3]: cada $\hat{s}$ debe reconocer su texto entre los del lote y cada texto su $\hat{s}$. El checkpoint se preentrenó así (`openvljepa.losses.bidirectional_infonce`, $\tau = 0{,}07$). Con un banco cerrado hay que

adaptar dos cosas, y en ambas se eligió la versión más cercana al original.

4.1 Dirección predictor → texto

Los negativos son **todas** las frases del banco, no las que casualmente cayeron en el lote. Es la misma InfoNCE con el conjunto de negativos exacto, y es lo que el decodificador evalúa en inferencia:

$$\mathcal{L}_{\rightarrow} = \frac{1}{B}\sum_{i=1}^{B} -\log \frac{\exp(\langle \hat{s}_i,\, s_{c_i}\rangle/\tau)}{\sum_{j=1}^{21}\exp(\langle \hat{s}_i,\, s_j\rangle/\tau)}$$

Se usa el banco completo (21) y no solo las frases del atributo consultado: los negativos de otros atributos son fáciles y aportan poco gradiente, pero obligan a que ŝ dependa de la consulta. La lectura en inferencia sí se restringe a las frases del atributo.

4.2 Dirección texto → predictor

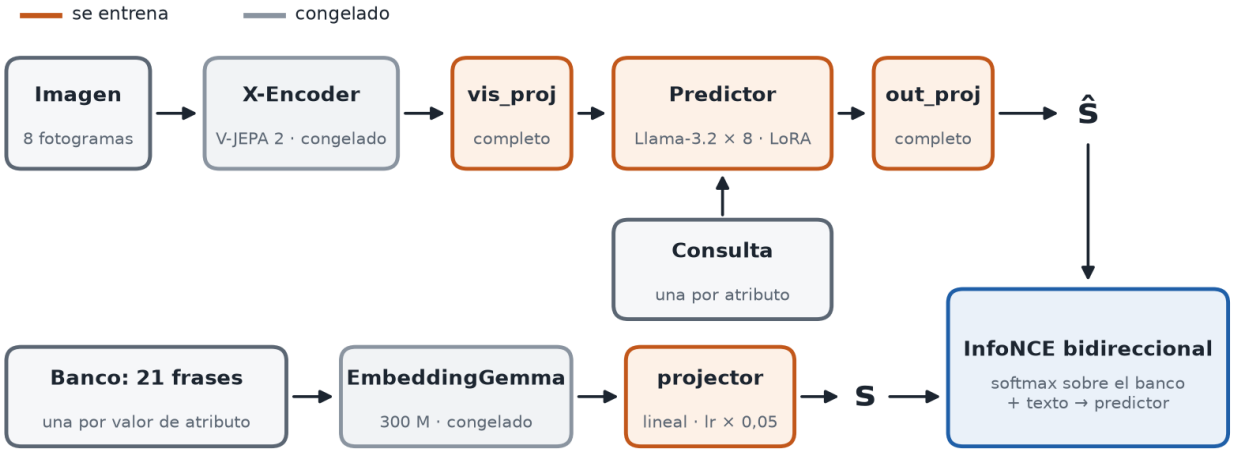
En el paper cada texto tiene una sola predicción positiva. Aquí varias imágenes del lote comparten la misma frase, así que el positivo es el conjunto de predicciones con esa frase (forma multi-positiva, como la contrastiva supervisada [12]). Con textos todos distintos coincide exactamente con la del paper (hay un test que lo comprueba):

$$\mathcal{L}_{\leftarrow} = \frac{1}{|J|}\sum_{j\in J} -\log \frac{\sum_{i\,:\,c_i=j}\exp(\langle \hat{s}_i,\, s_j\rangle/\tau)}{\sum_{i=1}^{B}\exp(\langle \hat{s}_i,\, s_j\rangle/\tau)}, \qquad \mathcal{L} = \tfrac{1}{2}(\mathcal{L}_{\rightarrow} + \mathcal{L}_{\leftarrow})$$

J son las frases presentes en el lote. Este término es el que da gradiente «desde el texto»: empuja cada referencia hacia sus predicciones y lejos de las demás, que es justo lo que separa las frases pegadas cuando la proyección de texto es entrenable. InfoNCE se mantiene, y no una pérdida de regresión (coseno, L1, L2), porque es la única con término anticolapso y por eso la única que el paper considera compatible con un Y-Encoder entrenable [1, §4.8 y Tabla 7c]; su descomposición en alineamiento y uniformidad es la de Wang e Isola [16].

- **τ = 0,07**, la del preentrenamiento del checkpoint. No se hace aprendible para no añadir un grado de libertad sin control; scripts/info_loss.py reporta la cota también con τ calibrada.
- **Consecuencia práctica:** el término inverso usa las predicciones de todo el lote, así que el micro-lote debe ser igual al lote efectivo (64) y un OOM no se puede resolver partiendo el lote. Si ocurriera, la ablación objective.symmetric=false sí admite micro-lotes.
- **Qué se registra:** la pérdida de entrenamiento y la de validación que se guardan son el término directo, porque es el que se interpreta como cota: $I(\hat{s}; \text{frase}) \geq \log 21 - L_{\rightarrow}$.

5. Qué se entrena y qué queda congelado



Componente	Parámetros	Decisión	Motivo	Ref.
X-Encoder (V-JEPA 2 ViT-L)	≈ 300 M	Congelado	El paper lo congela. La forma llega a 1,000 y el análisis del equipo ubica la pérdida después de ŝ: no es el cuello de botella. Entrenarlo sería lo más caro en memoria.	[1 §3.1], [10]
vis_proj (1024 → 2048)	2,10 M	Completo, lr 2e-5	Sin cambio respecto al run publicado: es el conector multimodal, como el proyector de los VLM.	[7], [8]
Predictor (8 capas Llama-3.2-1B)	LoRA r = 16	QLoRA, lr 2e-4	Sin cambio: es el presupuesto común de los cuatro modelos (base NF4, adaptadores de rango bajo en atención y MLP).	[7], [8]
out_proj (2048 → 1536)	3,15 M	**Completo, lr 2e-5** (antes congelado)	Es la capa que escribe ŝ en el espacio compartido. Si las referencias cambian de lugar, la salida del predictor debe poder seguirlas; congelada, solo LoRA río arriba puede compensar.	[1 §3.1]
Transformer del Y-Encoder (EmbeddingGemma-300M)	≈ 300 M	Congelado	El paper lo entrena con 2 000 M de pares; aquí hay 21 frases. Ajustarlo entero con ese vocabulario lo sobreajustaría y borraría el espacio de lenguaje que justifica usar VL-JEPA. Además no cabe con el resto en una GPU compartida.	[1 §3.1, T7b]
Proyección del Y-Encoder (768 → 1536)	1,18 M	**Completa, lr 1e-5** (antes congelada)	Es el lado de texto mínimo que puede separar referencias pegadas. El paper ablaciona exactamente esta configuración («trainable linear projection on top of frozen text encoder») y con EmbeddingGemma conserva el VQA (42,5) aunque pierde en clasificación y recuperación abiertas. Esta tarea es VQA discriminativo.	[1 T7e]

El multiplicador 0,05. El paper barre el multiplicador de lr del Y-Encoder: 0,05 es el mejor, 1,00 y 0,01 degradan, y 0,00 (congelado) es el peor, con −7,3 puntos en clasificación y −4,3 en recuperación [1, Tabla 7b]. El run publicado estaba en ese peor caso. Aquí la proyección usa 0,05 × 2e-4 = 1e-5.

Riesgo que se acepta. Una proyección lineal de 768 a 1536 dimensiones puede, en principio, mover 21 puntos a cualquier parte; con lr bajo y pocas épocas se queda cerca del preentrenamiento, pero el vínculo con el lenguaje se debilita. Se controla con la ablación de texto congelado (sección 7) y mirando en `bank_geometry.json` cuánto se movieron las referencias.

6. Épocas, presupuesto y selección

Con una consulta por imagen y por paso, el lote sigue teniendo 64 muestras y el costo por paso es el del run publicado. La consulta se elige con una rotación determinista: un desplazamiento fijo por (semilla, id de imagen) más la época, módulo 4. En cada época los cuatro atributos reciben un cuarto de las imágenes, y tras cuatro épocas cada imagen respondió exactamente una vez cada atributo.

Run	Épocas	Pasos	Pasadas de imagen	Supervisión por atributo	Uso
Comparable	1	1 954	125 000 (= los otros modelos)	≈ 31 250 imágenes	Fila de la tabla principal: mismo presupuesto de cómputo.
Extendido	4	7 816	500 000 (4×)	125 000 imágenes	Se reporta aparte, rotulado. Responde si al modelo le faltaba presupuesto.

- **Por qué más épocas aquí y no en los VLM.** El mejor paso del run publicado fue 1800 de 1954 y la curva no muestra saturación, a diferencia de LLaVA-OV (800) e InternVL3 (1000). Además el paper atribuye buena parte del desempeño a un preentrenamiento masivo que este checkpoint no tuvo: sin él pierde 21,7 puntos en clasificación [1, Tabla 7a].
- **Por qué no mezclarlo en la tabla principal.** El presupuesto común (configs/train/budget.yaml) es la definición operativa de comparación justa. Las épocas no se pueden fijar en el archivo del modelo: van por línea de comandos y quedan en el exp_id (…_e4).
- **Lo que no cambia:** lote 64, lr 2e-4 con coseno y 3 % de calentamiento (sobre el total de pasos), r = 16, NF4, semilla 0, selección por forma + posición en las mismas 500 imágenes de val, test una sola vez al final.
- **Evaluación en val:** cada 200 pasos a 1 época y cada 400 a 4 épocas. En val se hacen las cuatro consultas de cada imagen.

7. Cómo se evalúa bajo el nuevo contexto

7.1 Pérdida de información por etapa

`scripts/info_loss.py` carga un modelo (con o sin adaptador), extrae la media de tokens del X-Encoder y $\hat{s}$ para 10 000 imágenes de train y 2 000 de val, y escribe una fila por atributo con la información en cada etapa (nats), la pérdida entre etapas y la exactitud de la sonda frente a la del decodificador. Las sondas y τ se ajustan en train y todo se mide en val; test no se usa.

Columna	Significado
info_visual, info_predictor	V-información lineal del atributo en el X-Encoder y en $\hat{s}$.
info_decoder (y …_tau_train)	Cota InfoNCE a través del banco, con τ calibrada y con la del entrenamiento.
loss_decoder	info_predictor − info_decoder: lo que está en $\hat{s}$ y el banco no deja leer.
bank_capacity_nats	Máximo que un predictor ideal pasaría por ese banco (propiedad del banco).
acc_predictor, acc_decoder	Exactitud de la sonda sobre $\hat{s}$ y de la lectura por el banco.

Funciona con los dos bancos. Con el de captions marginaliza el softmax sobre las frases que comparten el valor del atributo, así que el run publicado y el v3 quedan en la misma escala.

7.2 Qué resultado confirmaría o refutaría el diseño

Si se observa…	Entonces…
En el run publicado, loss_decoder alto en y y capacidad del banco baja en y	Se confirma el mecanismo de la sección 2.2 con datos del proyecto.

Si se observa…	Entonces…
En v3, loss_decoder ≈ 0 en los cuatro atributos y y sube hacia el nivel de x	El rediseño resuelve el problema. La comparación con los VLM pasa a depender del presupuesto.
En v3, loss_decoder ≈ 0 pero info_predictor de y sigue bajo	El límite se movió al predictor: probar la ablación (c) y más rango de LoRA antes que más épocas.
La ablación de texto congelado iguala a v3	La proyección entrenable no era necesaria: conviene dejarla congelada y conservar el espacio de lenguaje original.
4 épocas no supera a 1	El presupuesto no era la causa; el paso 1800 del run publicado era ruido de val.

7.3 Runs de la cola

```
SCRIPT=scripts/train.py GPU=1 scripts/launch.sh s2c_vljepa \
    scripts/run_queue.sh configs/queues/stage2c_vljepa.txt

0) humo, 20 pasos              model=vljepa_fac train.max_steps=20 ...
1) comparable, 1 época         model=vljepa_fac
2) extendido, 4 épocas         model=vljepa_fac train.epochs=4 train.eval_every=400
3) semilla 1                   ... seed=1
4a) texto congelado            ... lora.text_modules=[]
4b) solo término directo       ... objective.symmetric=false
4c) out_proj congelado         ... lora.connector_modules=[predictor.vis_proj]
```

Cada línea entrena, elige el checkpoint en val y evalúa en las 10 000 imágenes de test. El humo va primero: es lo único que confirma que la configuración cabe en la GPU del cluster.

8. Cambios en el código

Archivo	Cambio
src/vlmfid/models/bank.py (nuevo)	Banco factorizado: frases, consultas, lectura explicable, rotación de consultas, cota del margen, capacidad y reporte de geometría. No carga modelos.
src/vlmfid/train/jepa.py (nuevo)	JEPAObjective: reúne lo de VL-JEPA que estaba repartido en el bucle (tabla de objetivos, predicción, pérdidas, lado de texto entrenable).
src/vlmfid/train/loop.py	El bucle delega en JEPAObjective; fija la época en el featurizer; refresca el banco antes de evaluar. La ruta de los VLM no cambia.
src/vlmfid/train/objectives.py	bank_logits, reverse_bank_infonce_loss y symmetric_bank_infonce_loss.
src/vlmfid/train/collate.py	JEPAFeaturizer con modo por atributo (una consulta al entrenar, cuatro al validar).
src/vlmfid/train/lora.py	lora.text_modules, grupo de lr «text», y comprobación de que el X-Encoder no se entrena.
src/vlmfid/train/config.py	Claves permitidas nuevas, validaciones y exp_id con «qattr».
src/vlmfid/models/vljepa.py	bank.kind, encode_pooled / project_pooled, refresh_bank, inferencia por atributo.
src/vlmfid/eval/infoloss.py, scripts/info_loss.py (nuevos)	Estimadores y script de pérdida de información por etapa.
configs/models/vljepa_fac.yaml, configs/train/models/vljepa_fac.yaml, configs/queues/stage2c_vljepa.txt (nuevos)	Configuración y cola del ajuste v3.
tests/test_vljepa_bank.py, tests/test_infoloss.py (nuevos), tests/test_train_e2e.py, tests/_tiny_vlm.py	Tests unitarios y de punta a punta con un OpenVLJEPA diminuto.

Compatibilidad. `configs/models/vljepa.yaml`, `configs/train/models/vljepa.yaml` y `configs/train/budget.yaml` no se tocaron: la configuración resuelta y el exp_id del run publicado son idénticos (hay un test), así que sus runs siguen siendo reanudables y reproducibles. Lo nuevo usa otro nombre de modelo (`vljepa_open_msrvtt_fac`) y por tanto otros exp_id.

Archivos borrados: ninguno. No hizo falta eliminar ni mover nada. La ruta anterior (banco de captions, negativos del lote o del banco) se conserva porque reproduce el run publicado y es el término de comparación.

9. Limitaciones

- **Sin validación a escala real.** Los tests usan pesos aleatorios en CPU: verifican que todo se conecta (qué se entrena, qué queda intacto, reanudación idéntica, recarga del adaptador), no que el modelo mejore.
- **Memoria.** La InfoNCE simétrica no admite partir el lote. El run publicado ya usaba 64 en un solo forward, y v3 agrega solo ~4,3 M de parámetros entrenables, pero hay que confirmarlo con el humo.
- **La comparación con los VLM cambia de forma.** VL-JEPA responde cuatro preguntas cerradas y el caption se compone; los VLM generan la frase. Es legítimo como «uso nativo de cada arquitectura», pero debe decirse en el informe. El vocabulario cerrado da a VL-JEPA una ventaja de formato que los VLM no tienen.
- **Atajo color–posición.** En M3DI el color depende causalmente de la posición horizontal [2]: adivinar la categoría solo con x acierta 42,5 %. Consultas separadas no impiden que el predictor use ese atajo. El split composicional existente no lo cubre (retira combinaciones forma × posición).
- **Las fronteras de color siguen siendo las de la evaluación.** El 22 % de las imágenes está a menos de 5° de un corte de categoría; ese techo afecta a todos los modelos por igual y no se aborda aquí.
- **Frases y consultas fijas.** Se eligieron una vez, sin ajustarlas con datos. Otra redacción puede dar otra geometría inicial; la proyección entrenable reduce esa dependencia, no la elimina.
- **Checkpoint.** Es una reimplementación abierta entrenada con MSR-VTT [9], no el modelo del paper. Un resultado bajo no refuta la arquitectura.

Referencias

[1] D. Chen et al. «VL-JEPA: Joint Embedding Predictive Architecture for Vision-language». arXiv:2512.10942, 2025. §3 (arquitectura y pérdida), §4.2 (VQA discriminativo), §4.8 y Tabla 7 (ablaciones).

[2] I. Daunhawer et al. «Identifiability Results for Multimodal Contrastive Learning». ICLR 2023, arXiv:2303.09166. Dataset Multimodal3DIdent.

[3] A. van den Oord, Y. Li, O. Vinyals. «Representation Learning with Contrastive Predictive Coding». arXiv:1807.03748, 2018. InfoNCE y su cota $I \geq \log N - L$.

[4] B. Poole et al. «On Variational Bounds of Mutual Information». ICML 2019, arXiv:1905.06922.

[5] Y. Xu et al. «A Theory of Usable Information under Computational Constraints». ICLR 2020, arXiv:2002.10689. V-información.

[6] A. Radford et al. «Learning Transferable Visual Models From Natural Language Supervision» (CLIP). ICML 2021, arXiv:2103.00020.

[7] E. Hu et al. «LoRA: Low-Rank Adaptation of Large Language Models». ICLR 2022, arXiv:2106.09685.

[8] T. Dettmers et al. «QLoRA: Efficient Finetuning of Quantized LLMs». NeurIPS 2023, arXiv:2305.14314.

[9] open-vljepa, reimplementación abierta de VL-JEPA: github.com/dion-jy/open-vljepa; pesos en Hugging Face cun-bjy/open-vljepa (Stage B, MSR-VTT).

[10] M. Assran et al. «V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning». arXiv:2506.09985, 2025.

[11] H. S. Vera et al. «EmbeddingGemma: Powerful and Lightweight Text Representations». arXiv:2509.20354, 2025.

[12] P. Khosla et al. «Supervised Contrastive Learning». NeurIPS 2020, arXiv:2004.11362.

[13] T. Cover, J. Thomas. «Elements of Information Theory», 2.ª ed., Wiley, 2006. Desigualdad de procesamiento de datos (teorema 2.8.1).

[14] G. Alain, Y. Bengio. «Understanding intermediate layers using linear classifier probes». arXiv:1610.01644, 2016.

[15] K. Ethayarajh. «How Contextual are Contextualized Word Representations?». EMNLP 2019, arXiv:1909.00512. Anisotropía de los espacios de representación de texto.

[16] T. Wang, P. Isola. «Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere». ICML 2020, arXiv:2005.10242.

Documentos internos: docs/ETAPA2_FINETUNING.md, docs/ETAPA2B_REFORMULACION.md, reports/tables/comparacion_test/, reports/tables/eda_finetuning/. Los PDF de [1], [2], [6]–[8], [10] y [11] están en docs/papers/.